Authentication and Access Control

Assignment Tasks 1 to 5: code screenshots and outputs

Task 1: Create a simple login form for a mock Zomato admin panel that asks for username and password, then displays 'Access Granted' only if both fields match hardcoded admin credentials.

A simple HTML login form with username and password fields. It shows 'Access Granted' only when both match the hardcoded credentials (admin / zomato@123); otherwise it shows 'Access Denied'.

Code (login.html)

```html
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Zomato Admin Panel - Login</title>
<style>
  body { font-family: Arial, sans-serif; background: #fff5f6; margin: 0; }
  .box { width: 340px; margin: 50px auto; background: #fff; padding: 28px;
         border-radius: 10px; border: 1px solid #f0d0d4; }
  h2 { color: #e23744; text-align: center; margin-top: 0; }
  input { width: 100%; padding: 10px; margin: 8px 0 14px; box-sizing: border-box;
          border: 1px solid #ccc; border-radius: 5px; }
  button { width: 100%; padding: 11px; background: #e23744; color: #fff;
           border: none; border-radius: 5px; font-size: 15px; cursor: pointer; }
  #msg { text-align: center; font-weight: bold; margin-top: 16px; }
</style>
</head>
<body>
<div class="box">
  <h2>Zomato Admin Login</h2>
  <label>Username</label>
  <input type="text" id="username" placeholder="Enter username">
  <label>Password</label>
  <input type="password" id="password" placeholder="Enter password">
  <button onclick="login()">Login</button>
  <div id="msg"></div>
</div>

<script>
  // Hardcoded admin credentials (for demo only)
  const ADMIN_USER = "admin";
  const ADMIN_PASS = "zomato@123";

  function login() {
    const u = document.getElementById("username").value;
    const p = document.getElementById("password").value;
    const msg = document.getElementById("msg");

    if (u === ADMIN_USER && p === ADMIN_PASS) {
      msg.style.color = "green";
      msg.innerText = "Access Granted";
    } else {
      msg.style.color = "red";
      msg.innerText = "Access Denied";
    }
  }
</script>
</body>
</html>
```

Output



Login form *Correct credentials* *Wrong password*

Task 2:
Write a Python function check_access(user_role, resource) that returns True if the user_role is allowed to access the resource, otherwise False. Test it with roles like 'customer', 'restaurant_owner', and resources like 'menu_edit', 'order_history','admin_dashboard'.

Hint: Use a dictionary to map roles to allowed resources.

A dictionary maps each role to the resources it may access. The function returns True only if the resource is in that role's list; unknown roles get False.

Code (access_control.py)

```python
# Task 2: Role-based access control

# Dictionary mapping each role to the resources it may access
ROLE_PERMISSIONS = {
    "customer": ["order_history"],
    "restaurant_owner": ["menu_edit", "order_history"],
    "admin": ["menu_edit", "order_history", "admin_dashboard"],
}


def check_access(user_role, resource):
    """Return True if user_role is allowed to access resource, else False."""
    return resource in ROLE_PERMISSIONS.get(user_role, [])


# ---- Test cases ----
tests = [
    ("customer", "menu_edit"),
    ("customer", "order_history"),
    ("customer", "admin_dashboard"),
    ("restaurant_owner", "menu_edit"),
    ("restaurant_owner", "order_history"),
    ("restaurant_owner", "admin_dashboard"),
    ("admin", "admin_dashboard"),
    ("guest", "order_history"),
]

for role, res in tests:
    print(f"check_access('{role}', '{res}') -> {check_access(role, res)}")
```

Output

```
python access_control.py

check_access('customer', 'menu_edit') -> False
check_access('customer', 'order_history') -> True
check_access('customer', 'admin_dashboard') -> False
check_access('restaurant_owner', 'menu_edit') -> True
check_access('restaurant_owner', 'order_history') -> True
check_access('restaurant_owner', 'admin_dashboard') -> False
check_access('admin', 'admin_dashboard') -> True
check_access('guest', 'order_history') -> False
```

Task 3:
Simulate a two-factor authentication flow like Paytm: after a user enters the correct password, generate a 6-digit OTP and display it in the console. Ask the user to input the OTP and display 'Login Successful' only if it matches.

Code (two_factor.py)

```python
# Task 3: Two-factor authentication (Paytm style)
import random

USERNAME = "rahul"
PASSWORD = "paytm@123"


def generate_otp():
    return str(random.randint(100000, 999999))  # 6-digit OTP


username = input("Enter username: ")
password = input("Enter password: ")

if username == USERNAME and password == PASSWORD:
    otp = generate_otp()
    print(f"[Paytm] Your OTP is: {otp}")
    entered = input("Enter the OTP: ")
    if entered == otp:
        print("Login Successful")
    else:
        print("Invalid OTP. Login Failed")
else:
    print("Incorrect username or password")
```

Output (correct OTP, wrong OTP, wrong password)

```
python two_factor.py

Enter username: rahul
Enter password: paytm@123
[Paytm] Your OTP is: 943380
Enter the OTP: 943380
Login Successful

Enter username: rahul
Enter password: paytm@123
[Paytm] Your OTP is: 206644
Enter the OTP: 000000
Invalid OTP. Login Failed

Enter username: rahul
Enter password: wrong
Incorrect username or password
```

Task 4: Given an array of user objects with properties: username, isPremium, and lastLogin, write a function that returns only those users who are premium and have logged in within the la days.

Constraint: Assume lastLogin is a date string in 'YYYY-MM-DD' format, and today is '2024-06-10'.

The function filters users who are premium and whose lastLogin is within 7 days of today (2024-06-10). Expected result: aman, sneha and vikram. priya is not premium and karan last logged in too long ago.

Code (premium_users.js)

```js
// Task 4: Premium users who logged in within the last 7 days
const TODAY = new Date("2024-06-10");

function getActivePremiumUsers(users) {
  return users.filter(user => {
    const diffDays = (TODAY - new Date(user.lastLogin)) / (1000 * 60 * 60 * 24);
    return user.isPremium && diffDays >= 0 && diffDays <= 7;
  });
}

// ---- Test data ----
const users = [
  { username: "aman",   isPremium: true,  lastLogin: "2024-06-08" },
  { username: "priya",  isPremium: false, lastLogin: "2024-06-09" },
  { username: "karan",  isPremium: true,  lastLogin: "2024-05-20" },
  { username: "sneha",  isPremium: true,  lastLogin: "2024-06-03" },
  { username: "vikram", isPremium: true,  lastLogin: "2024-06-10" },
];

console.log(getActivePremiumUsers(users));
```

Output

```
node premium_users.js

[
  { username: 'aman', isPremium: true, lastLogin: '2024-06-08' },
  { username: 'sneha', isPremium: true, lastLogin: '2024-06-03' },
  { username: 'vikram', isPremium: true, lastLogin: '2024-06-10' }
]
```

Task 5: Use ChatGPT to generate a list of 5 common access control mistakes developers make in real-world apps like Instagram or Flipkart, and for each, write a one-line fix or prevention tip.

Mistake	Real-world example	One-line fix
1. Broken Object Level Authorization (IDOR)	Flipkart: changing /orders/1234 to /orders/1235 shows another customer's order.	Always check that the logged-in user owns the requested record on the server.
2. Relying only on frontend checks	Instagram: hiding the 'Delete' or 'Admin' button in the UI while the API still accepts the request.	Enforce every permission on the backend; treat the UI as cosmetic only.
3. Missing function-level access control	Flipkart: a normal user can call /admin/refund because the endpoint has no role check.	Add role checks (RBAC) to every endpoint and deny access by default.

Mistake	Real-world example	One-line fix
4. Trusting client-sent roles or IDs	Instagram: sending "role":"admin" or another user_id in the request body or token.	Read role and user ID from the verified session/JWT, never from user input.
5. Over-permissive or never-expiring sessions/tokens	Instagram: an old token or a shared link keeps working after logout or a password change.	Use short-lived tokens, revoke them on logout, and follow least privilege.